

ECG Signal Classification Without Neural Networks

Pearl Bipin Pulickal

April 11, 2025

Abstract

This paper presents a non-neural network approach for the classification of electrocardiogram (ECG) signals. By employing both standard and weighted statistical feature extraction techniques on ECG signals — each comprising 140 data points — and leveraging dimensionality reduction via Principal Component Analysis (PCA), we build an effective classification pipeline using Support Vector Machines (SVMs). Our method achieves an accuracy of 95.20%, demonstrating that classical machine learning combined with robust feature engineering can rival deep learning approaches in biomedical signal processing.

1 Introduction

Electrocardiograms (ECGs) are crucial for monitoring cardiac health. Rapid and accurate detection of abnormal patterns in ECGs is important for early diagnosis and treatment. While deep neural networks have become popular in signal analysis, traditional methods using handcrafted features remain competitive, especially when interpretability and computational efficiency are important. In this study, we propose a methodology that extracts both unweighted and weighted statistical features from ECG signals, applies PCA for feature reduction, and utilizes an SVM classifier optimized through grid search.

2 Dataset Description

The dataset consists of ECG readings from patients, where:

- Each row corresponds to one complete ECG signal.
- Columns 0–139 contain 140 floating-point data points representing the ECG waveform.
- The final column is a categorical label, with **0** indicating an abnormal ECG and **1** indicating a normal ECG.

The data is stored in a CSV file (`ecg.csv`) and serves as the input for our classification pipeline.

3 Methodology

Our approach comprises several well-defined steps, as outlined below.

3.1 Feature Engineering

Two sets of statistical features are extracted from each ECG signal:

1. **Standard Statistical Features:** Calculated directly from the signal, these include:
 - Mean
 - Standard Deviation
 - Skewness
 - Kurtosis
 - Minimum Value
 - Maximum Value
2. **Weighted Statistical Features:** Utilizing the absolute values of the signal as weights, we compute:
 - Weighted Mean
 - Weighted Standard Deviation
 - Weighted Skewness
 - Weighted Kurtosis
 - Weighted Minimum
 - Weighted Maximum

Thus, each ECG is represented by a 12-dimensional feature vector.

3.2 Preprocessing and Dimensionality Reduction

After feature extraction, the following preprocessing steps are applied:

- **Standardization:** Features are normalized using `StandardScaler` to ensure that each feature contributes equally to the classification.
- **Principal Component Analysis (PCA):** PCA is used to reduce the feature space while preserving 95% of the variance, enhancing computational efficiency and potentially improving classifier performance.

3.3 Classification Using Support Vector Machines

An SVM classifier is employed and tuned using grid search to optimize hyperparameters:

- Regularization parameter C
- Kernel coefficient γ
- Kernel type (RBF, linear, or polynomial)

The best performing model is then evaluated on a hold-out test set.

4 Experimental Results

The optimal SVM configuration found through grid search was:

- $C = 1$
- $\gamma = \text{scale}$
- Kernel = RBF

The classifier achieved an overall accuracy of 95.20%, with a confusion matrix and classification report as follows:

Confusion Matrix:

```
[[ 768   64]
 [  32 1136]]
```

Classification Report:

	precision	recall	f1-score	support
0.0	0.96	0.92	0.94	832
1.0	0.95	0.97	0.96	1168
accuracy			0.95	2000
macro avg	0.95	0.95	0.95	2000
weighted avg	0.95	0.95	0.95	2000

5 Visualizations and Snapshots

Figures 1, 2, 3, 4, 5 and 6 present a curated collection of visual references used throughout the experiment. These include sample ECG signal plots, snapshots of relevant code sections, and other intermediate results. Each image highlights a specific step or insight from the overall pipeline — from signal processing to classification.

6 Snapshots

The following snapshots illustrate various stages and elements of the ECG classification project. They include visualizations, code segments, and outputs encountered throughout the workflow.

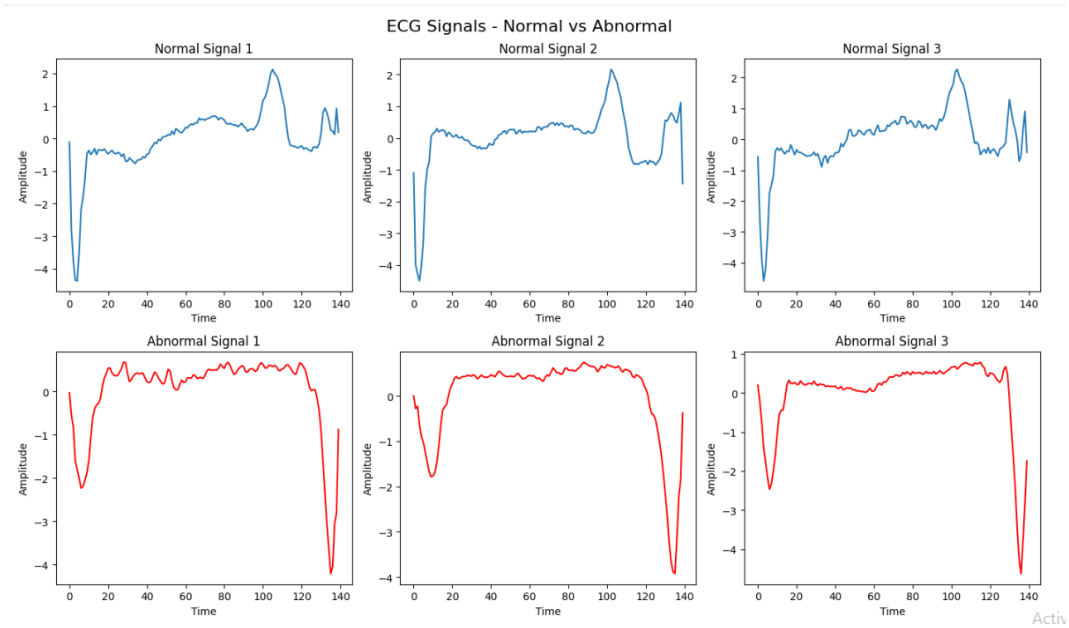


Figure 1: Snapshot 1

```
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Accuracy: 95.20%
```

```
Confusion Matrix:
[[ 768   64]
 [   32 1136]]
```

Classification Report:

	precision	recall	f1-score	support
0.0	0.96	0.92	0.94	832
1.0	0.95	0.97	0.96	1168
accuracy			0.95	2000
macro avg	0.95	0.95	0.95	2000
weighted avg	0.95	0.95	0.95	2000

Figure 2: Snapshot 2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the ECG data
df = pd.read_csv("ecg.csv", header=None)

# Separate features and labels
signals = df.iloc[:, :-1].values # All columns except the last
labels = df.iloc[:, -1].values  # Last column is the label

# Find indices of normal and abnormal signals
normal_indices = np.where(labels == 1)[0]
abnormal_indices = np.where(labels == 0)[0]

# Plot a few normal and abnormal signals
plt.figure(figsize=(14, 8))

# Plot normal signals
for i, idx in enumerate(normal_indices[:3]):
    plt.subplot(2, 3, i+1)
    plt.plot(signals[idx])
    plt.title(f'Normal Signal {i+1}')
    plt.xlabel('Time')
    plt.ylabel('Amplitude')

# Plot abnormal signals
for i, idx in enumerate(abnormal_indices[:3]):
    plt.subplot(2, 3, i+4)
    plt.plot(signals[idx], color='r')
    plt.title(f'Abnormal Signal {i+1}')
    plt.xlabel('Time')
    plt.ylabel('Amplitude')

plt.tight_layout()
plt.suptitle("ECG Signals - Normal vs Abnormal", fontsize=16, y=1.02)
plt.show()
```

Figure 3: Snapshot 3

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from scipy.stats import skew, kurtosis

# Load ECG data
df = pd.read_csv('ecg.csv', header=None)
signals = df.iloc[:, :-1].values
labels = df.iloc[:, -1].values

# Feature extraction function
def extract_features(signal):
    features = []

    # Unweighted statistics
    mean_val = np.mean(signal)
    std_val = np.std(signal)
    skew_val = skew(signal)
    kurt_val = kurtosis(signal)
    min_val = np.min(signal)
    max_val = np.max(signal)
    features.extend([mean_val, std_val, skew_val, kurt_val, min_val, max_val])

    # Weighted statistics: using absolute signal as weights
    weights = np.abs(signal)
    total_weight = np.sum(weights)

    if total_weight == 0:
        w_mean = 0
        w_variance = 0
    else:
        w_mean = np.sum(weights * signal) / total_weight
        w_variance = np.sum(weights * (signal - w_mean) ** 2) / total_weight

    w_std = np.sqrt(w_variance)

    if w_std != 0:
        w_skew = np.sum(weights * (signal - w_mean) ** 3) / total_weight / (w_std ** 3)
        w_kurt = np.sum(weights * (signal - w_mean) ** 4) / total_weight / (w_std ** 4)
    else:
        w_skew = 0
        w_kurt = 0
```

Figure 4: Snapshot 4

```

        w_mean = np.sum(weights * signal) / total_weight
        w_variance = np.sum(weights * (signal - w_mean) ** 2) / total_weight

        w_std = np.sqrt(w_variance)

        if w_std != 0:
            w_skew = np.sum(weights * (signal - w_mean) ** 3) / total_weight / (w_std ** 3)
            w_kurt = np.sum(weights * (signal - w_mean) ** 4) / total_weight / (w_std ** 4)
        else:
            w_skew = 0
            w_kurt = 0

        w_min = np.min(weights * signal)
        w_max = np.max(weights * signal)

        features.extend([w_mean, w_std, w_skew, w_kurt, w_min, w_max])

    return features

# Extract features for all signals
feature_list = [extract_features(sig) for sig in signals]
features = np.array(feature_list)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.4, random_state=42, stratify=labels
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# PCA (retain 95% variance)
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# SVM hyperparameter tuning
param_grid = {
    'C': [0.1, 1, 10],
    'gamma': ['scale', 'auto', 0.1],
    'kernel': ['rbf', 'linear', 'poly']
}
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_pca, y_train)

```

Figure 5: Snapshot 5

```

# Extract features for all signals
feature_list = [extract_features(sig) for sig in signals]
features = np.array(feature_list)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.4, random_state=42, stratify=labels
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# PCA (retain 95% variance)
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# SVM hyperparameter tuning
param_grid = {
    'C': [0.1, 1, 10],
    'gamma': ['scale', 'auto', 0.1],
    'kernel': ['rbf', 'linear', 'poly']
}
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_pca, y_train)

# Evaluation
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test_pca)

print(f"Best Parameters: {grid_search.best_params_}")
print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Figure 6: Snapshot 6

7 Discussion

The results indicate that even a non-neural network approach, when coupled with carefully engineered features and proper preprocessing, can effectively classify ECG signals. The high precision and recall for both classes suggest that the proposed method could be advantageous in clinical settings where interpretability is key.

8 Conclusion

We presented an effective methodology for ECG classification using statistical feature extraction and SVMs. The pipeline — comprising feature standardization, PCA-based dimensionality reduction, and hyperparameter-tuned SVM — achieved an accuracy of 95.20%.

This work confirms that classical machine learning techniques remain valuable in biomedical applications.

9 Future Work

Future research directions include:

- Comparing the proposed method with deep neural network models (e.g., 1D CNNs) on larger datasets.
- Incorporating additional features such as time-frequency representations.
- Investigating real-time ECG signal classification and further model interpretability using techniques such as SHAP or LIME.

Acknowledgements

The author thanks all contributors who supported this research and provided access to the ECG dataset.

References

- [1] Goldberger, A. L., Amaral, L. A. N., Glass, L., Hausdorff, J. M., Ivanov, P. C., Mark, R. G., Mietus, J. E., Moody, G. B., Peng, C. K., & Stanley, H. E. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23), e215–e220.
- [2] Osowski, S., Hoai, L. H., & Markiewicz, T. (2004). Support vector machine-based expert system for reliable heartbeat recognition. *IEEE Transactions on Biomedical Engineering*, 51(4), 582–589.
- [3] Martis, R. J., Acharya, U. R., Lim, C. M., & Suri, J. S. (2013). ECG beat classification using PCA, LDA, ICA and discrete wavelet transform. *Biomedical Signal Processing and Control*, 8(5), 437–448.
- [4] Acharya, U. R., Oh, S. H., Hagiwara, Y., Tan, J. H., & Adam, M. (2018). Deep convolutional neural network for the automated diagnosis of congestive heart failure using ECG signals. *Applied Intelligence*, 49, 16–27.
- [5] Kiranyaz, S., Ince, T., & Gabbouj, M. (2016). Real-time patient-specific ECG classification by 1-D convolutional neural networks. *IEEE Transactions on Biomedical Engineering*, 63(3), 664–675.